

Product Requirements Document

MERN Stack Task Tracker Web Application

1. Overview

This document outlines the product requirements for a full-stack Task Tracker web application built using the MERN stack (MongoDB, Express.js, React.js, Node.js). The application is designed to demonstrate proficiency in full-stack development, REST API design, database management, and modern React fundamentals. It provides users with a seamless interface to manage their daily tasks efficiently.

2. Technology Stack

- **Frontend:** React.js (Hooks, Functional Components, Context API or Redux for state)
- **Backend:** Node.js with Express.js framework
- **Database:** MongoDB (accessed via Mongoose ODM)
- **Deployment Strategy:** Frontend hosted on Vercel/Netlify; Backend hosted on Render/Heroku; Database hosted on MongoDB Atlas.

3. Core Features & Mandatory Requirements

- **CRUD Operations:** Users must be able to Create new tasks, Read (View) existing tasks in a list format, Update task details or status, and Delete tasks entirely.
- **Form Validation:** Strict frontend and backend validation for task inputs (e.g., title is required, title length limits, status must match defined enums).
- **REST APIs:** Properly structured, stateless RESTful API endpoints for all database interactions.
- **Dynamic UI Updates:** The frontend must reflect all database changes (additions, updates, deletions) immediately without requiring a full page refresh, leveraging React's reactive state management.
- **Responsive Design:** The user interface must be fully accessible and visually appealing on mobile phones, tablets, and desktop computers.

4. Bonus & Extended Features

- **Filtering & Sorting:** Users can filter tasks by status (e.g., Pending, In Progress, Completed) and sort them by creation date or custom priority levels.
- **Toast Notifications:** Implementation of non-intrusive toast notifications to alert users of successful operations (e.g., "Task created successfully") or backend errors.
- **Reusable Architecture:** Highly modular React component structure, extracting UI elements like Buttons, Modals, and Input Fields into reusable components.
- **Environment Variables:** Secure configuration management using `.env` files to store sensitive data like MongoDB URIs, API base URLs, and ports.

5. Database Schema (Task Model)

Field	Data Type	Constraints / Notes
title	String	Required, Max 100 characters
description	String	Optional, Max 500 characters
status	String	Enum: ['Pending', 'In Progress', 'Completed'], Default: 'Pending'
priority	String	Enum: ['Low', 'Medium', 'High'], Default: 'Medium' (Bonus Feature)
createdAt	Date	Auto-generated system timestamp
updatedAt	Date	Auto-updates on modification

6. REST API Endpoints

Method	Endpoint	Description
GET	/api/tasks	Fetch all tasks (supports optional query parameters for filtering and sorting)
POST	/api/tasks	Create and save a new task
PUT	/api/tasks/:id	Update an existing task's fields by ID
DELETE	/api/tasks/:id	Delete a specific task by ID